

Lab 04 - Building Spring Boot RESTful Web Services and Spring Data JPA

1. Introduction

Imagine you're a new fresher Spring Boot developer of IT department in a University. Your manager has asked you to develop a RESTful API for Orchid Management to manage just a simple information of an orchid.

Orchid (OrchidID, OrchidName, isNatural, orchidDescription, orchidCategory, isAttractive, orchidURL)

The RESTful API has to support adding, viewing, modifying, and removing information for an orchid - a standardized usage action verbs better known as Create, Read, Update, Delete (CRUD).

This lab explores creating a RESTful API using **Spring Boot 3** with Data Source is MS SQL Server. An **SQL Server Database** will be created to persist the orchid data (reading and managing) by **Spring Data JPA**

2. Lab Objectives

In this lab, you will:

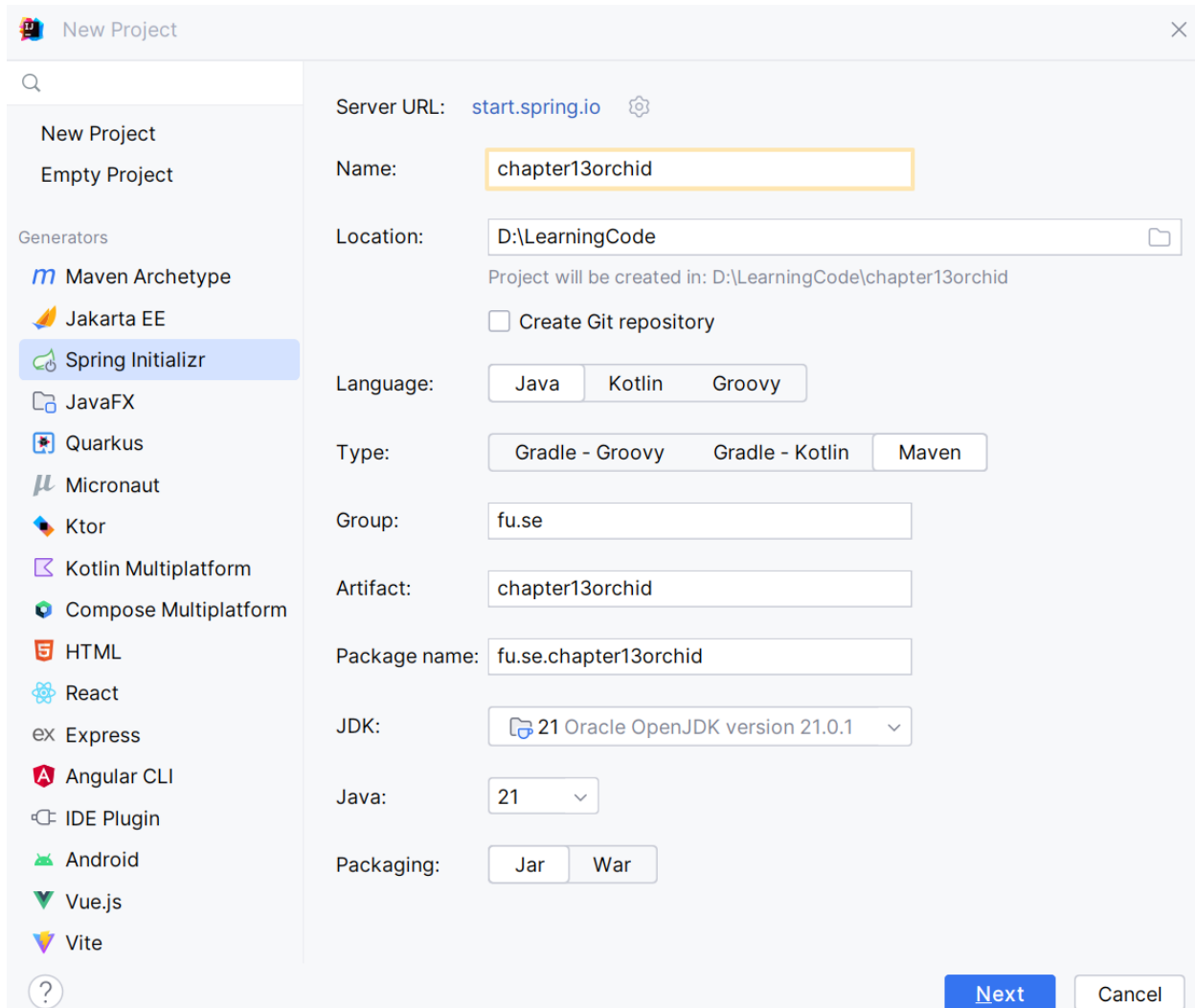
- Use the **IntelliJ IDEA** to create Spring Initializr Project (Maybe you can create the Spring Initializr project in <https://start.spring.io/> then open this project in IntelliJ IDEA.
- Choose **Spring Web** for Building web, including *RESTful*, applications using Spring MVC. Uses Apache Tomcat as the default embedded container. **Spring Data JPA SQL** for persisting data in SQL stores with Java Persistence API using Spring Data and Hibernate. **MS SQL Server Driver SQL** - A JDBC and R2DBC driver that provides access to Microsoft SQL Server and Azure SQL Database from any Java application.
- Use in SQL Server Database as a Data Source

- Develop 3-Layer with Repository Architecture to perform CRUD actions using RESTful API with Spring Boot 3.
- Run the project and test the application actions.

Activity 01: Design the Orchid Management

Step 01. Open IntelliJ IDEA, File | New | Initializr Project

In the left pane of the New Project wizard, select Spring Initializr Project.

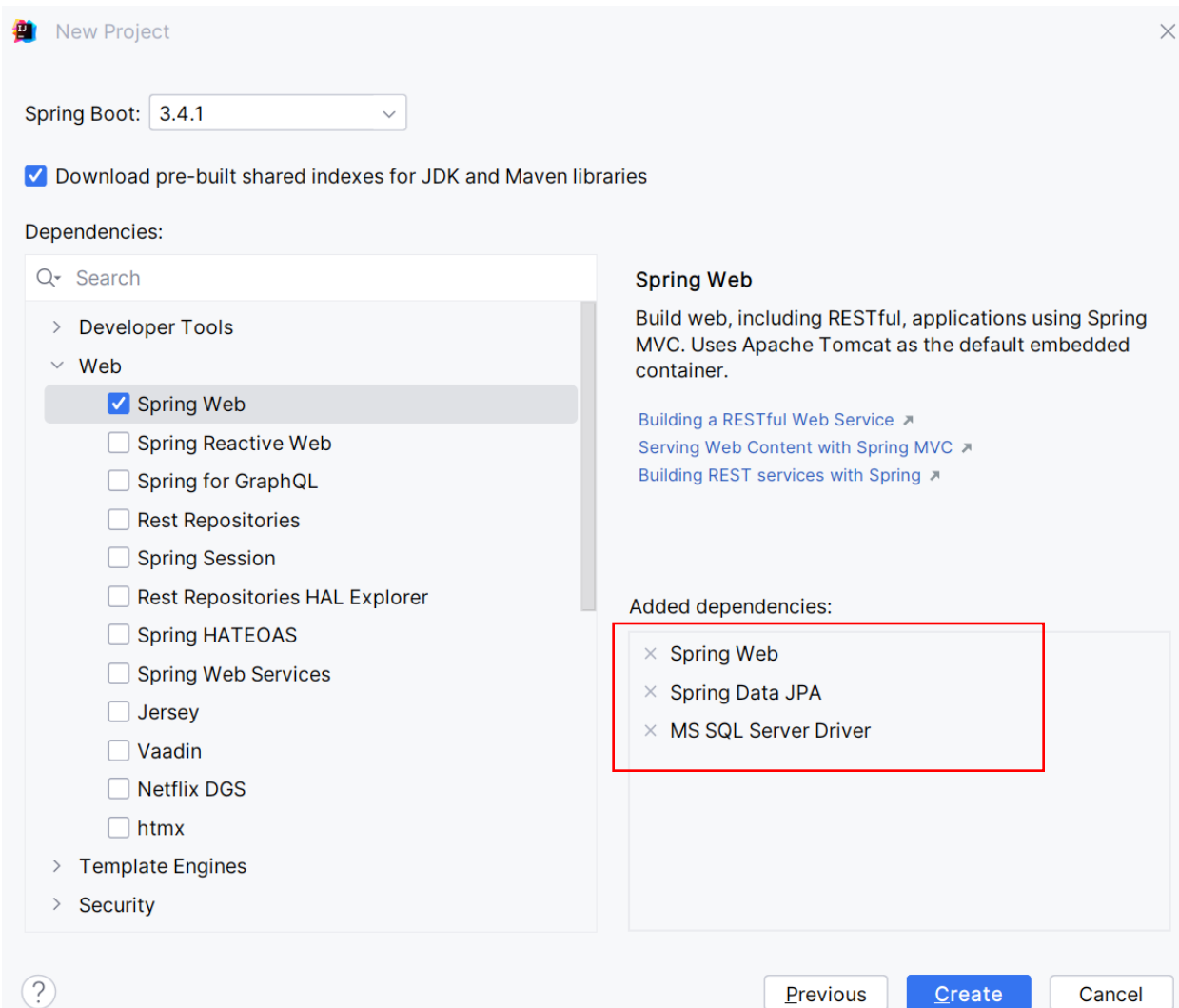


The screenshot shows the 'New Project' dialog in IntelliJ IDEA. On the left, under 'Generators', 'Spring Initializr' is selected. The right pane shows the following configuration:

- Server URL: start.spring.io
- Name:
- Location:
Project will be created in: D:\LearningCode\chapter13orchid
- ☐ Create Git repository
- Language: ☒ Java ☐ Kotlin ☐ Groovy
- Type: ☒ Gradle - Groovy ☐ Gradle - Kotlin ☐ Maven
- Group:
- Artifact:
- Package name:
- JDK:
- Java:
- Packaging: ☒ Jar ☐ War

At the bottom right, there are 'Next' and 'Cancel' buttons.

Step 02. Fill the information → Next



Step 03. Choose the dependency/dependencies → Click Create

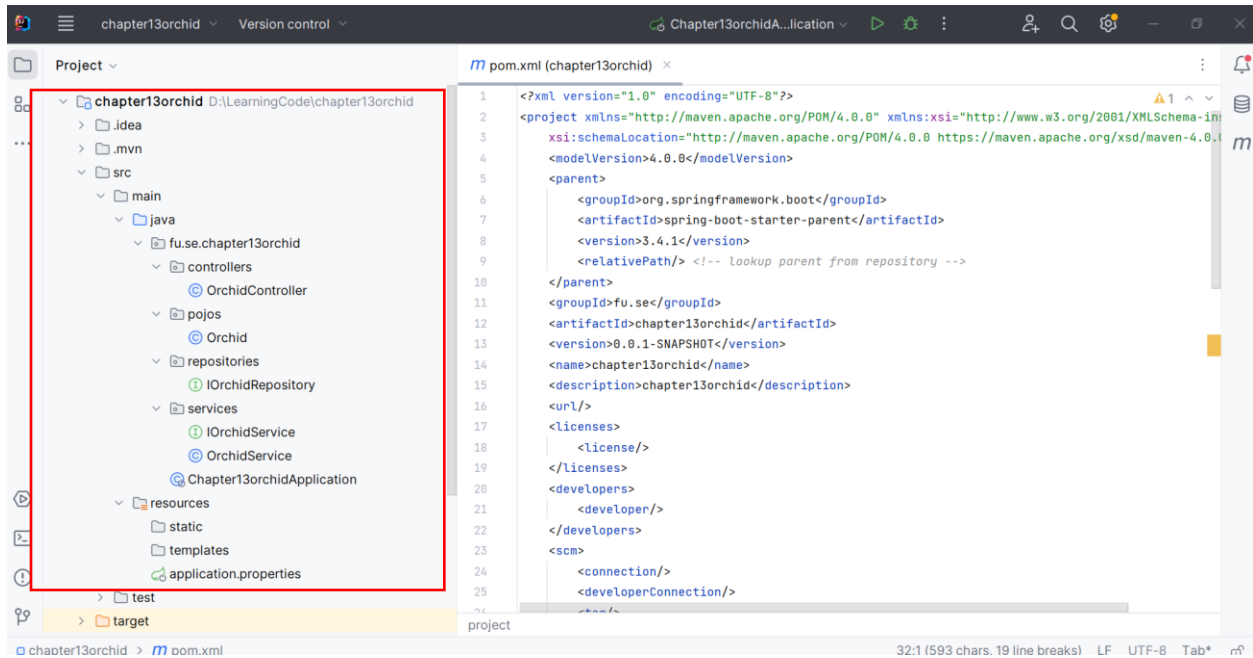
Or you can using Spring Initializr project in <https://start.spring.io/> then open this project in IntelliJ IDEA

Step 04. Edit pom.xml, add the dependencies for

- MS SQL Server Driver **mssql-jdbc**
- Spring Web - RESTful API: **spring-boot-starter-web**
- Spring Data JPA for Paging and Sort data: **spring-data-jpa**
- Testing **spring-boot-starter-test**

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>com.microsoft.sqlserver</groupId>
    <artifactId>mssql-jdbc</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Step 05. The structure of the Lab project



Step 06. The application.properties file



Activity 02: Implement RESTful API Orchid Management with CRUD functions

Step 01. Create **pojos** package, then create Orchid Entity

Entity Class: Represents a database table

Annotations:

- **@Entity**: Marks the class as a JPA entity
- **@Id**: Specifies the primary key field
- **@GeneratedValue**: Strategy for generating primary key values
- **@Column**: Maps a field to a database column

```
Orchid.java x
7  @Entity
8  @Table(name = "orchid")
9  public class Orchid implements Serializable {
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     @Column(name = "orchid_id")
13     private int orchidID;
14     @Column(name = "orchid_name")
15     private String orchidName;
16     @Column(name = "is_natural", columnDefinition = "bit default 0")
17     private boolean isNatural;
18     @Column(name = "orchid_description")
19     private String orchidDescription;
20     @Column(name = "orchid_category")
21     private String orchidCategory;
22     @Column(name = "is_attractive", columnDefinition = "bit default 0")
23     private boolean isAttractive;
24     @Column(name = "orchid_url")
25     private String orchidURL;
```

Step 02. Implement Repository by creating **repositories** package, IOrchidRepository interface

- Create JPA repository interface, this interface extending JpaRepository
- Provides CRUD operations and more
- No implementation needed (Spring Data JPA provides it)

```
I IOrchidRepository.java x
1 package fu.se.chapter13orchid.repositories;
2
3 import fu.se.chapter13orchid.pojos.Orchid;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface IOrchidRepository extends JpaRepository<Orchid, Integer> {
7 }
```

Step 03. Implement Service Layer includes: IOrchidService interface, OrchidService class in **services** package.

- Define service interfaces: Create interfaces to define the business logic for application.
- Implement service classes: Implement the service interfaces and use the repositories to interact with the database.
- Use the IOrchidRepository to interact with the Repository (Data Layer)

```

1 IOrchidService.java x
2
3 import fu.se.chapter13orchid.pojos.Orchid;
4
5 import java.util.List;
6 import java.util.Optional;
7
8 3 usages 1 implementation
9 public interface IOrchidService {
10     1 usage 1 implementation
11     public List<Orchid> getAllOrchids();
12     1 usage 1 implementation
13     public Orchid insertOrchid(Orchid orchid);
14     1 usage 1 implementation
15     public Orchid updateOrchid(int orchidID, Orchid orchid);
16     1 usage 1 implementation
17     public void deleteOrchid(int orchid);
18
19     1 usage 1 implementation
20     public Optional<Orchid> getOrchidByID (int orchidID);
21 }

```

```

1 OrchidService.java x
2
3 package fu.se.chapter13orchid.services;
4
5 import fu.se.chapter13orchid.pojos.Orchid;
6 import fu.se.chapter13orchid.repositories.IOrchidRepository;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.stereotype.Service;
9
10 import java.util.List;
11 import java.util.Optional;
12
13 @Service
14 public class OrchidService implements IOrchidService{
15     @Autowired
16     private IOrchidRepository iOrchidRepository;
17     1 usage
18     @Override
19     public List<Orchid> getAllOrchids() {
20         return iOrchidRepository.findAll();
21     }
22     1 usage
23     @Override
24     public Orchid insertOrchid(Orchid orchid) {
25         return iOrchidRepository.save(orchid);
26     }
27 }

```

```

23      @Override
24      public Orchid updateOrchid(int orchidID, Orchid orchid) {
25          Orchid o = iOrchidRepository.getById(orchidID);
26          if(o != null) {
27              o.setOrchidName(orchid.getOrchidName());
28              o.setOrchidDescription(orchid.getOrchidDescription());
29          }
30          return null;
31      }
32      1 usage
33      @Override
34      public void deleteOrchid(int orchid) { iOrchidRepository.deleteById(orchid); }
35      1 usage
36      @Override
37      public Optional<Orchid> getOrchidByID(int orchidID) { return iOrchidRepository.findById(orchidID); }
40  }

```

Step 04. Implement REST Controller

@RequestMapping - Maps HTTP requests to specific controller methods based on the request path, HTTP method, and other criteria. It can be used at both the class and method level. Attributes of **@RequestMapping**:

- value: Specifies the request path.
- method: Specifies the HTTP method (e.g., RequestMethod.GET, RequestMethod.POST).
- params: Specifies request parameters that must be present.
- headers: Specifies request headers that must be present.
- consumes: Specifies the content type of the request body.
- produces: Specifies the content type of the response body.

The annotations are shortcuts for **@RequestMapping** with specific HTTP methods. They simplify the code and improve readability.

- **@GetMapping**
- **@PostMapping**
- **@PutMapping**
- **@DeleteMapping**
- **@PatchMapping**

@PathVariable: Extracts values from URI path variables and binds them to method parameters.

@RequestParam - Extracts values from query parameters and binds them to method parameters.

@RequestParam attributes:

- value: Specifies the name of the query parameter.
- required: Indicates whether the parameter is required.
- defaultValue: Specifies a default value if the parameter is not present.

```

OrchidController.java x
1  package fu.se.chapter13orchid.controllers;
2
3  import ...
12
13  @RestController
14  @CrossOrigin
15  @RequestMapping("/orchids")
16  public class OrchidController {
17      @Autowired
18      private IOrchidService iOrchidService;
19
20      // HTTP Verb GET, POST, PUT, DELETE
21      @GetMapping("/")
22      public ResponseEntity<List<Orchid>> fetchAll(){...}
25
26      @PostMapping("/")
27      @ResponseStatus(HttpStatus.CREATED)
28      public Orchid saveOrchid(@RequestBody Orchid orchid) { return iOrchidService.insertOrchid(orchid); // 201 Created }
31
32      @PutMapping("/{id}")
33      public ResponseEntity<Orchid> updateOrchid(@PathVariable int id, @RequestBody Orchid o) {...}
37
38      @DeleteMapping("/{id}")
39      public ResponseEntity<String> deleteOrchid(@PathVariable int id) {...}
43
44      @GetMapping("/{id}")
45      public ResponseEntity<Optional<Orchid>> getById(@PathVariable int id){...}
49  }
    
```

The details of action methods in REST Controller

```

@GetMapping("/")
public ResponseEntity<List<Orchid>> fetchAll(){
    return ResponseEntity.ok(iOrchidService.getAllOrchids()); // 200 OK
}
    
```

```

@GetMapping("/{id}")
public ResponseEntity<Optional<Orchid>> get0ById(@PathVariable int id){
    Optional<Orchid> o = iOrchidService.get0rchidByID(id);
    return ResponseEntity.ok(o); // 200 OK
}

@PostMapping("/")
@ResponseStatus(HttpStatus.CREATED)
public Orchid save0rchid(@RequestBody Orchid orchid){
    return iOrchidService.insert0rchid(orchid); // 201 Created
}

@PutMapping("/{id}")
public ResponseEntity<Orchid> update0rchid(@PathVariable int id, @RequestBody Orchid o) {
    Orchid update0rchid = iOrchidService.update0rchid(id, o);
    return ResponseEntity.ok(update0rchid);
}

@DeleteMapping("/{id}")
public ResponseEntity<String> delete0rchid(@PathVariable int id) {
    iOrchidService.delete0rchid(id);
    return ResponseEntity.ok( body: "Deleted!!");
}

```

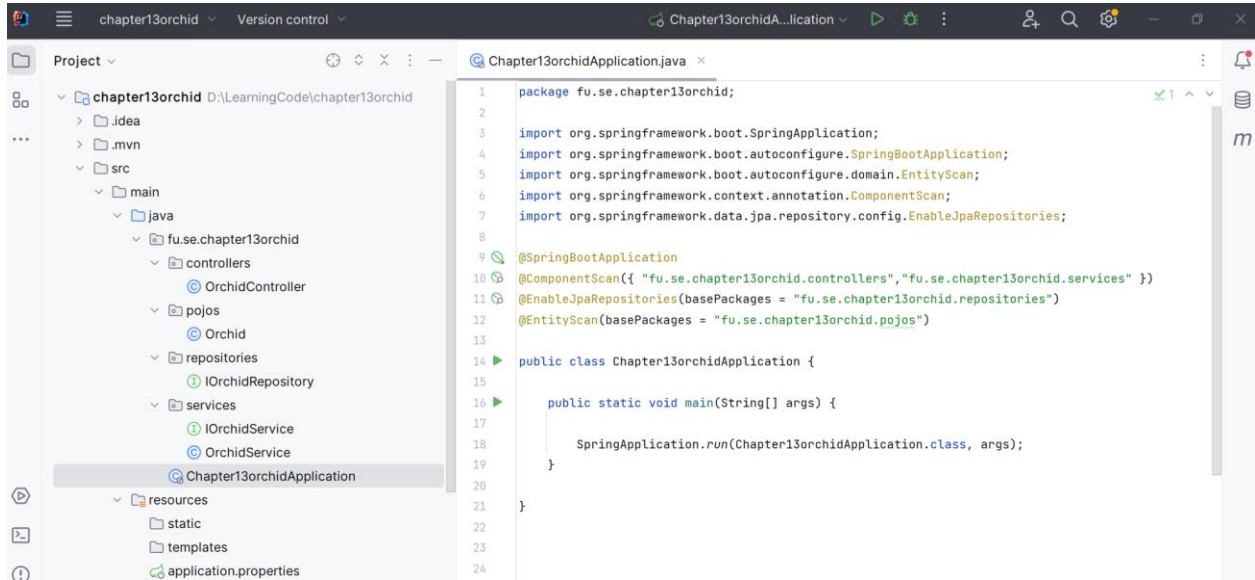
Step 05. The main entry point for Spring Boot application

The `@SpringBootApplication` annotation is a convenience annotation in Spring Boot that combines several annotations to simplify the configuration and initialization of a Spring Boot application.

When you use `@SpringBootApplication`, it automatically includes the following annotations:

- **@EnableAutoConfiguration:** This annotation tells Spring Boot to automatically configure your application based on the libraries on the classpath. For example, if Spring Boot detects that you have a database driver in your project, it will automatically configure a datasource.
- **@ComponentScan:** This tells Spring to scan the current package and its sub-packages for annotated components (such as `@Component`, `@Service`, `@Repository`, and `@Controller`) to register them as Spring beans.

- **@Configuration:** This marks the class as a source of bean definitions for the application context, meaning that you can define @Bean methods in this class.



```
1 package fu.se.chapter13orchid;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.boot.autoconfigure.domain.EntityScan;
6 import org.springframework.context.annotation.ComponentScan;
7 import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
8
9 @SpringBootApplication
10 @ComponentScan({ "fu.se.chapter13orchid.controllers", "fu.se.chapter13orchid.services" })
11 @EnableJpaRepositories(basePackages = "fu.se.chapter13orchid.repositories")
12 @EntityScan(basePackages = "fu.se.chapter13orchid.pojos")
13
14 public class Chapter13orchidApplication {
15
16     public static void main(String[] args) {
17
18         SpringApplication.run(Chapter13orchidApplication.class, args);
19     }
20
21 }
22
23
24
```

Activity 03: Run and test the RESTful API

Step 01. Run the RESTful API Project

```

2025-01-02T04:15:33.162+07:00 INFO 24436 --- [chapter13orchid] [main] org.hibernate.Version : HHH000412: Hibernate ORM core ver
2025-01-02T04:15:33.234+07:00 INFO 24436 --- [chapter13orchid] [main] o.h.c.internal.RegionFactoryInitiator : HHH000026: Second-level cache dis
2025-01-02T04:15:33.822+07:00 INFO 24436 --- [chapter13orchid] [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignorin
2025-01-02T04:15:33.881+07:00 INFO 24436 --- [chapter13orchid] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2025-01-02T04:15:34.603+07:00 INFO 24436 --- [chapter13orchid] [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection (
2025-01-02T04:15:34.606+07:00 INFO 24436 --- [chapter13orchid] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2025-01-02T04:15:34.850+07:00 WARN 24436 --- [chapter13orchid] [main] org.hibernate.orm.deprecation : HHH90000025: SQLServerDialect doe
2025-01-02T04:15:34.892+07:00 INFO 24436 --- [chapter13orchid] [main] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
Database driver: undefined/unknown
Database version: 16.0
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2025-01-02T04:15:36.442+07:00 INFO 24436 --- [chapter13orchid] [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform avail
2025-01-02T04:15:37.239+07:00 INFO 24436 --- [chapter13orchid] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFact
2025-01-02T04:15:37.751+07:00 WARN 24436 --- [chapter13orchid] [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enable
2025-01-02T04:15:38.698+07:00 INFO 24436 --- [chapter13orchid] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (htt
2025-01-02T04:15:38.721+07:00 INFO 24436 --- [chapter13orchid] [main] f.s.c.Chapter13orchidApplication : Started Chapter13orchidApplicatio
  
```

Step 02. Test the RESTful API using Postman

	API name	HTTP Method	Path	Status Code
1	GET Orchids	GET	http://localhost:8080/orchids/	200 OK
2	POST Orchid	POST	http://localhost:8080/orchids/	201 Created
3	DELETE Orchid	DELETE	http://localhost:8080/orchids/{id}	204 No Content
4	PUT Orchid	PUT	http://localhost:8080/orchids/{id}	200 OK
5	GET Orchid	GET	http://localhost:8080/orchids/{id}	20 OK

Step 02. Test the GET method (for getting all orchids information)

The screenshot shows the REST Client interface. On the left, a sidebar lists collections, environments, and history. A red box highlights the 'RESTful API with Orchids Java' collection, which contains several endpoints. The main panel shows a GET request to 'http://localhost:8080/orchids/'. Below the request bar, there are tabs for Params, Auth, Headers (7), Body, Scripts, and Settings. The 'Query Params' section is empty. The 'Body' tab is selected, showing a successful response with status '200 OK', 58 ms, and 1.49 KB. The response body is displayed in a 'Pretty' format, showing a JSON array of orchid objects. A red box also highlights the request bar and the 'Send' button.

GET `http://localhost:8080/orchids/` **Send**

Params Auth Headers (7) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body **200 OK** - 58 ms - 1.49 KB **Save Response**

Pretty Raw Preview Visualize JSON

```

1 [
2   {
3     "orchidID": 6,
4     "orchidName": "Primulinum26",
5     "orchidDescription": "Delta Description",
6     "orchidCategory": "Delta",
7     "orchidURL": "https://lh3.googleusercontent.com/pw/
  AP1GczMqaC0bLlBkZaZcZsr9P0g7F97dzVed13d71kUWxmQzpYHfeEiLmaRmeYfG15Jy4J54QE7h8mb92Uj
  Xmt1EdPAEF7goFHRkdhQy8CiQXc05moJec-jSTr88PpCiQvS_1QLY2LrK1vLmxt8tIauV=w760-h889-s
  -no-gm",
8     "natural": true,
9     "attractive": true
  }
  ]
  
```

Step 03. Test the POST method for creating a new orchid

The screenshot shows the Postman interface for a RESTful API. The request is a POST to `http://localhost:8080/orchids/`. The body is a JSON object with the following fields: `orchidName`, `orchidDescription`, `orchidCategory`, `orchidURL`, `attractive`, and `natural`. The response is a 201 Created status with a response time of 250 ms and a body of 652 B. The response body is a JSON object with the field `orchidID` set to 1008.

```

1 {
2   "orchidName": "Phalaenopsis 2",
3   "orchidDescription": "Phalaenopsis Orchids Desc",
4   "orchidCategory": "Phalaenopsis Orchids",
5   "orchidURL": "https://lh3.googleusercontent.com/pw/
  AP1GczMqaCobl1BkZaAzcz9Pd7F97dzVedi3d7ikUwXmQzpYHfEilMaRmeYfG1S3y4J54QF7h8mb9zJX
  mt1EdPAEf7goFHRwkdhQy8CiqXc05moJec-jSTr88PpCiQvS_1QLY2LrK1vvLmxtk8TlauV=w768-h889-s-n
  o-gm",
6   "attractive": true,
7   "natural": true
8 }
  
```

```

1 {
2   "orchidID": 1008,
  }
  
```

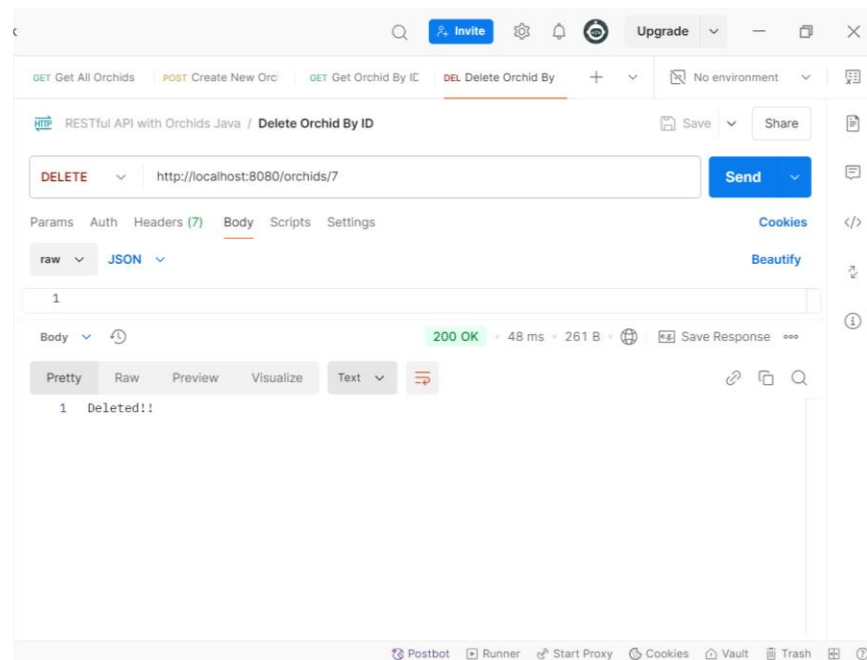
Step 04. Test the GET method for an existing orchid.

The screenshot shows the Postman interface for a RESTful API. The request is a GET to `http://localhost:8080/orchids/9`. The response is a 200 OK status with a response time of 94 ms and a body of 639 B. The response body is a JSON object with the following fields: `orchidID`, `orchidName`, `orchidDescription`, `orchidCategory`, `orchidURL`, `natural`, and `attractive`.

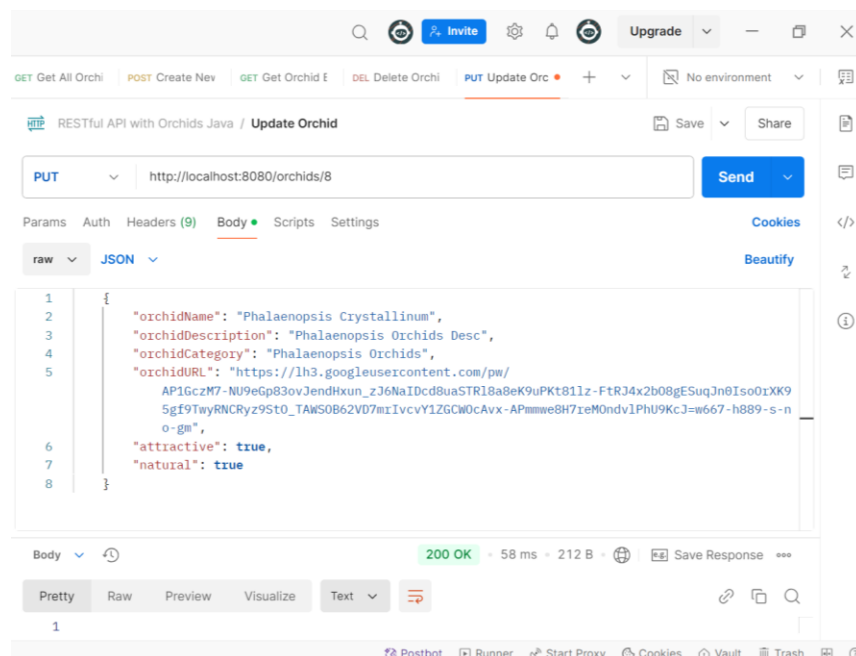
```

1 {
2   "orchidID": 9,
3   "orchidName": "Mini Yaya",
4   "orchidDescription": "Phalaenopsis Orchids Desc",
5   "orchidCategory": "Phalaenopsis Orchids",
6   "orchidURL": "https://lh3.googleusercontent.com/pw/
  AP1GczPKHw-7L3gD4DEyxII6_H0tpzf1a-YNv3fWLVTLNkahT21VJrboRHT5xPahT20fxK0TiAWjcyzvdPviEkD
  Bma38wYbBZ9--uQYYX0p_9qIGjRIA0r1x-0Yess-H5-SmHpanoa5KVYyd1Pn-FEP06I=w667-h889-s-no-gm",
7   "natural": true,
8   "attractive": true
9 }
  
```

Step 05. Test the DELETE method for an existing orchid.



Step 06. Test the PUT method for an existing orchid.



Note 1. CrudRepository vs. JpaRepository interface

@NoRepositoryBean

public interface **CrudRepository**<T,ID> extends Repository<T,ID>

<https://docs.spring.io/spring-data/commons/docs/current/api/org/springframework/data/repository/CrudRepository.html>

@NoRepositoryBean

public interface **JpaRepository**<T,ID> extends ListCrudRepository<T,ID>, **ListPagingAndSortingRepository**<T,ID>, QueryByExampleExecutor<T>

<https://docs.spring.io/spring-data/jpa/docs/current/api/org/springframework/data/jpa/repository/JpaRepository.html>

Note 2. Quan hệ One-to-One, One-to-Many, Many-to-Many trong JPA (bidirection)

Quan hệ One to Many (1 Department có nhiều Employee, 1 Employee chỉ thuộc về 1 Department)

```
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL,
orphanRemoval = true)
    private List<Employee> employees = new ArrayList<>();
    //Getters and Setters
}
```

```
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;
private String name;
@ManyToOne
@JoinColumn(name = "department_id")
private Department department;
//Constructors
//Getters, Setters
}
```

Quan hệ One to One (1 Employee có 1 Address, 1 Address chỉ thuộc về 1 Employee)

```
@Entity
public class Address {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "person_id")
    private Employee employee;

    private String city;

    //Constructors
    //Getters, Setters
}
```

```
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;
```

```
@OneToOne(mappedBy = "employee", cascade = CascadeType.ALL, fetch =
FetchType.LAZY)
private Address address;
//Constructors
//Getters, Setters
}
```

Quan hệ Many to Many (1 Student tham gia nhiều Course, 1 Course có nhiều Student tham gia)

```
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToMany(mappedBy = "students")
    private Set<Course> courses = new HashSet<>();
    //Constructors
    //Getters, Setters
}
```

```
@Entity
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToMany
    @JoinTable(
        name = "course_student",
        joinColumns = @JoinColumn(name = "course_id"),
        inverseJoinColumns = @JoinColumn(name = "student_id")
    )
}
```

```
)  
  
private Set<Student> students = new HashSet<>();  
  
//Constructors  
//Getters, Setters  
  
}
```